RAM.

- If the amount of RAM is between 2-8 GB, the ideal swap space should equal to the size of RAM
- If the amount of RAM is more than 8 GB, the ideal swap space should be 0.75 times the size of RAM

Let's now see how to improve the swap performance.

Ideally, dedicate a set of high performance disks spread across two or more controllers. If the swap resides on a busy disk, then, to reduce swap latency, it should be located as close to a busy partition (like */var*) as possible to reduce seek time for the drive head.

While using different partitions or hard disks of different speeds for swap, you can assign priorities to each partition so that the kernel will use the higher priority (actually high performance) disk first. You can set the priority using the *pri* option in your */etc/fstab* file. Here is an example of the */etc/fstab* file under such circumstances:

```
/dev/sda6       swap        swap        pri=4     0   0
/dev/sda8       swap        swap        pri=4     0   0
/dev/sdb3       swap        swap        pri=1     0   0
```

The kernel will first use the swap with a higher pri value—in our example, */dev/sda6* and */dev/sda8*. It will not use */dev/sdb3* until all of */dev/sda6* and */dev/sda8* are fully used. In addition, the kernel will distribute visit counts in a round-robin style across all the devices with equal priorities.

The basics of network tuning

First, let's look at what happens when a packet is transmitted out of the NIC (network interface card):

1. Writes data to socket file (goes in transmit buffer).
2. Kernel encapsulates data into protocol data units.
3. PDUs are moved to the per-device transmit queue.
4. Driver copies PDU from head of queue to NIC.
5. NIC raises the interrupt when PDU is transmitted.

Now let's see what actually happens when a packet is received on NIC:

1. NIC receives frame and uses DMA to copy into receive buffer.
2. NIC raises CPU interrupt.
3. Kernel handles interrupt and schedules a *softirq*.
4. The *softirq* is handled to move packets up to the IP layer for the routing decision.
5. If it's a local delivery, then the packet is decapsulated and placed into the socket's receive buffer.

The 'txqueuelen' value, as reported by the *ifconfig* command, is the length of the transmit queue on the NIC. Packets are scheduled by a queuing discipline. I ran the ifconfig command to find out its value in my system:

```
[root@legacy ~]# ifconfig
eth0    Link encap:Ethernet  HWaddr 00:15:C5:C0:B6:76
        inet addr:172.24.0.252  Bcast:172.24.255.255  Mask:255.255.0.0
        inet6 addr: fe80::215:c5ff:fec0:b676/64  Scope:Link
        UP BROADCAST MULTICAST  MTU:1500  Metric:1

        RX packets:27348 errors:0 dropped:0 overruns:0 frame:0
        TX packets:20190 errors:0 dropped:0 overruns:0 carrier:0

        collisions:0 txqueuelen:1000

        RX bytes:22661585 (21.6 MiB)  TX bytes:3683788 (3.5 MiB)
        Interrupt:17
```

As you can see, it's set to 1000, by default. Let's now issue another interesting command related to the previous 'txqueuelen' value:

```
[root@legacy ~]# tc -s qdisc show dev eth0

qdisc pfifo_fast 0: root bands 3 priomap  1 2 2 2 1 2 0 0 1 1 1 1 1 1 1 1

 Sent 3601538 bytes 20199 pkt (dropped 0, overlimits 0 requeues 0)

 rate 0bit 0pps backlog 0b 0p requeues 0
```

The above command basically outputs the 'qdisc' for eth0, and also notifies us whether the connection is dropping packets. If it is, then we need to increase the length of the queue. Here's how we can increase the value to 10001:

```
[root@legacy ~]# ip link set eth0 txqueuelen 10001
```

We verify whether the new value has taken effect by running the *ifconfig* command again:

```
[root@legacy ~]# ifconfig
eth0    Link encap:Ethernet  HWaddr 00:15:C5:C0:B6:76
        inet addr:172.24.0.252  Bcast:172.24.255.255  Mask:255.255.0.0
        inet6 addr: fe80::215:c5ff:fec0:b676/64  Scope:Link
        UP BROADCAST MULTICAST  MTU:1500  Metric:1

        RX packets:27501 errors:0 dropped:0 overruns:0 frame:0

        TX packets:20209 errors:0 dropped:0 overruns:0 carrier:0

        collisions:0 txqueuelen:10001

        RX bytes:22693532 (21.6 MiB)  TX bytes:3685904 (3.5 MiB)

        Interrupt:17
```

We can also control the size of the maximum send and receive transport socket buffers. We can use the following values in our */etc/sysctl.conf* file:

- *net.core.rmem_default* – the default buffer size used for the receive buffer